

# Experiment No. 7

## Lab Practical: Application Layer Packet Analysis using Packet Tracer Tool

### Aim:

To analyze Application Layer packets (DNS & HTTP) using a packet tracer tool (Wireshark) and understand their structure and working.

### Tools Required:

- Wireshark (Packet Analyzer)
- Internet Connection or Local DNS/HTTP Server
- Browser or Command Line Tools (nslookup, ping)

### Procedure:

#### Part A: DNS Packet Analysis

1. **Start Wireshark** and begin packet capture on the active network interface (WiFi/Ethernet).
2. **Open Command Prompt or Terminal** and type:
3. `nslookup www.google.com`
4. In Wireshark, stop the capture after response.
5. Apply filter:
6. `dns`
7. Select the DNS query and observe the following:
  - **Transaction ID**
  - **Flags:** Query/Response
  - **Query Name:** e.g., `www.google.com`
  - **Query Type:** A (IPv4)
8. Click on the **DNS Response** packet and observe:
  - **Resolved IP Address**
  - **TTL (Time to Live)**
  - **Answer Section**

#### Part B: HTTP Packet Analysis

1. In the browser, type:
2. `http://example.com`
3. In Wireshark, apply filter:
4. `http`
5. Observe the **HTTP GET** request:
  - **Method:** GET

- **Host:** example.com
- **User-Agent**
- 6. Observe the **HTTP Response**:
  - **Status Code:** 200 OK
  - **Content-Type:** text/html
  - **Content-Length**

### Observations:

| Field              | DNS Query                 | DNS Response            | HTTP Request | HTTP Response |
|--------------------|---------------------------|-------------------------|--------------|---------------|
| Protocol           | UDP                       | UDP                     | TCP          | TCP           |
| Port               | 53                        | 53                      | 80           | 80            |
| Method/Type        | Standard Query (A Record) | Response                | GET /        | 200 OK        |
| IP Address Queried | www.google.com            | e.g.,<br>192.178.211.84 | example.com  | 34.223.124.45 |
| TTL                | -                         | 75 seconds<br>(example) | -            | -             |
| Content-Type       | -                         | -                       | -            | text/html     |

### Result:

Successfully analyzed DNS and HTTP packets using Wireshark. The structure and fields of Application Layer packets were observed and interpreted.

### Conclusion:

- DNS resolves domain names to IP addresses via **UDP port 53**.
- HTTP works over **TCP port 80**, where the client sends a GET request and receives a response with status and content.
- Wireshark effectively captures and visualizes **Application Layer** communication for analysis and troubleshooting.

# Output / Screenshots

The screenshot displays the Wireshark interface with a packet capture of DNS traffic. The top pane shows a list of packets, with packet 202 selected. The middle pane shows the packet details for the selected packet, which is a DNS query response from 192.178.211.84 to 192.178.211.84. The bottom pane shows the raw packet data in hexadecimal and ASCII.

**Packet 202: Packet, 95 bytes on wire (768 bits), 95 bytes captured (768 bits) on interface DeviceNPF\_{64C3F248-933E-4583-A737-500E09772956}, Id 0**

**Ethernet II, Src: RuckusWirele 37:17:80 (c81c3285717:80), Dst: Dell\_0d:96:87 (00be43:8d:96:87)**

**Internet Protocol Version 4, Src: 10.17.60.103, Dst: 10.17.60.103**

**User Datagram Protocol, Src Port: 53, Dst Port: 54932**

**Domain Name System (response)**

Transaction ID: 0x933e

Flags: 0x0100 Standard query response, No error

1 ..... = Response: Message is a response

..... = Opcode: Standard query (0)

..... = Authoritative: Server is not an authority for domain

..... = Truncated: Message is not truncated

..... = Recursion desired: Do query recursively

..... = Recursion available: Server can do recursive queries

..... = Z: reserved (0)

..... = Answer authenticated: Answer/authority portion was not authenticated by the server

..... = Non-authenticated data: Unacceptable

..... = RCODE: Reply code: No error (0)

Questions: 1

Answer RRs: 1

Authority RRs: 0

Additional RRs: 0

Queries

accounts.google.com: type A, class IN

Name: accounts.google.com

Name Length: 19

Label Count: 3

Type: A (1) (Host Address)

Class: IN (0x0001)

Answers

accounts.google.com: type A, class IN, addr: 192.178.211.84

Name: accounts.google.com

Type: A (1) (Host Address)

Class: IN (0x0001)

Time to live: 75 (1 minute, 15 seconds)

Data length: 4

Address: 192.178.211.84

[Request ID: 100]

[Time: 1.438300 milliseconds]

Packets: 29743 - Displayed: 405 (1.6%) - Dropped: 0 (0.0%)

The screenshot displays the Wireshark interface with a packet capture of HTTP traffic. The top pane shows a list of packets, with packet 802 selected. The middle pane shows the packet details for the selected packet, which is an HTTP GET request from 10.17.60.103 to 192.178.211.84. The bottom pane shows the raw packet data in hexadecimal and ASCII.

**Packet 802: Packet, 867 bytes on wire (6936 bits), 867 bytes captured (6936 bits) on interface DeviceNPF\_{64C3F248-933E-4583-A737-500E09772956}, Id 1**

**Ethernet II, Src: RuckusWirele 37:17:80 (c81c3285717:80), Dst: Dell\_0d:96:87 (00be43:8d:96:87)**

**Internet Protocol Version 4, Src: 10.17.60.103, Dst: 192.178.211.84**

**Transmission Control Protocol, Src Port: 80, Dst Port: 16440, Seq: 1463, Len: 613**

**12 Reassembled TCP Segments (2273 bytes): #801(1400), #802(813)**

**Hypertext Transfer Protocol**

HTTP/1.1 200 OK\r\n

Response Version: HTTP/1.1

Status Code: 200

[Status Code Description: OK]

Response Phrase: OK

Date: Tue, 06 Oct 2020 04:39:14 GMT\r\n

Server: Apache/2.4.48 ((Ubuntu))\r\n

Upgrade: h2,h2c\r\n

Connection: Upgrade, Keep-Alive\r\n

Last-Modified: Wed, 29 Jun 2022 00:12:33 GMT\r\n

Tag: <?P=20200303>grip\r\n

Accept-Ranges: bytes\r\n

Vary: Accept-Encoding\r\n

Content-Encoding: gzip\r\n

Content-Length: 1900\r\n

[Content length: 1900]

Keep-Alive: timeout=5, max=100\r\n

Content-Type: text/html; charset=UTF-8\r\n

\r\n

[Request ID: 876]

[Time since request: 392.628200 milliseconds]

[Request URI: /]

[Full request URI: http://news.rsl.com/]

Content-encoding entity body (gzip): 1900 bytes -> 3961 bytes

File Date: 3961 bytes

Line-based text data: text/html (131 lines)

html\r\n

<html>\r\n

<title>NewsRSS - Connecting ... </title>\r\n

</html>\r\n

</body>\r\n

Packets: 29743 - Displayed: 405 (1.6%) - Dropped: 0 (0.0%)

# NeverSSL

## What?

This website is for when you try to open Facebook, Google, Amazon, etc on a wifi network, and nothing happens. Type "http://neverssl.com" into your browser's url bar, and you'll be able to log on.

The image shows a side-by-side comparison of network traffic and a web browser. On the left, Wireshark captures an HTTP GET request to 'http://neverssl.com/online/'. The packet details show the request line, status code 200, and various headers including 'Server: Apache/2.4.6.8', 'Upgrade: h2,h2c', and 'Keep-Alive: timeout=5, max=100'. The raw data section shows the hexadecimal and ASCII representation of the packet. On the right, a web browser window displays the 'NeverSSL' website, which has a blue header with the site name and a white body with the same 'What?' and 'How?' text as the introductory page.

**What?**

This website is for when you try to open Facebook, Google, Amazon, etc on a wifi network, and nothing happens. Type "http://neverssl.com" into your browser's url bar, and you'll be able to log on.

**How?**

neverssl.com will never use SSL (also known as TLS). No encryption, no strong authentication, no HSTS, no HTTP2.0, just plain old unencrypted HTTP and forever stuck in the dark ages of internet security.

**Why?**

Normally, that's a bad idea. You should always use SSL and secure encryption when possible. In fact, it's such a bad idea that most websites are now using https by default.

And that's great, but it also means that if you're relying on poorly-behaved wifi networks, it can be hard to get online. Secure browsers and websites using https make it impossible for those wifi networks to send you to a login or payment page. Basically, those networks can't tap into your connection just like attackers can't. Modern browsers are so good that they can remember when a website supports encryption and even if you type in the website name, they'll use https.

And if the network never redirects you to this page, well as you can see, you're not missing much.